

# **AI-Driven Multi-Modal Framework for Vulnerability Detection in Smart Contracts**

**BY**

***Group ID: 06***

***TG/2020/738 - N.G.N. Sansala***

***TG/2021/1067 - N.U.M. Hewage***

**A PROPOSAL SUBMITTED IN PARTIAL FULFILLMENT OF THE  
REQUIREMENTS FOR THE SUBJECT OF  
“RESEARCH PROJECT”  
DEPARTMENT OF INFORMATION AND COMMUNICATION  
TECHNOLOGY  
FACULTY OF TECHNOLOGY**

# AI-Driven MultiModal Framework for Vulnerability Detection in Smart Contracts

A Proposal Presented

By

*GROUP ID: 06*

*TG/2020/738 - N.G.N. Sansala*

*TG/2021/1067 - N.U.M. Hewage*

Submitted to

Department of Information and Communication Technology

Faculty of Technology

In partial fulfillment of the requirements for the

Degree of Bachelor of ICT Honors

Supervisor:

Ms. Akila Brahmana

## **Acknowledgement**

First and foremost, we wish to express our profound gratitude to our supervisor, Ms. Akila Brahmana, who spared their valuable time, inspiration, and motivation for this research project. Their vast experience and deep technical insights into blockchain security and artificial intelligence have been an outstanding contribution to identifying the research direction and enhancing the quality of our work.

We also extend our sincere appreciation to the Department of Information and Communication Technology, Faculty of Technology, for providing us with the required resources, facilities, and a research-based academic atmosphere favorable to innovation.

Furthermore, we acknowledge the global developer communities and data providers such as Etherscan, DAppSCAN, and the contributors of the SmartBugs and SolidiFI datasets. Their open-source data and comprehensive audit reports have enabled the construction of the required foundation for building our multimodal predictive models and the creation of the vulnerability alert system described in this proposal.

Lastly, we are truly thankful to our colleagues and families for their infinite support, encouragement, and inspiration throughout the initial stages of this project.

## Table of Contents

|                                                          |     |
|----------------------------------------------------------|-----|
| Acknowledgement.....                                     | ii  |
| List of Tables.....                                      | iii |
| List of Figures.....                                     | 3   |
| 1. Introduction.....                                     | 1   |
| 2. Background.....                                       | 2   |
| 3. Research Question and Research Problem Statement..... | 3   |
| 3.1 Statement of Research Problem.....                   | 3   |
| 3.2 Research Question.....                               | 3   |
| 4. Aim and Specific Objectives.....                      | 3   |
| 4. 1 Aim:.....                                           | 3   |
| 4. 2 Specific Objectives:.....                           | 4   |
| 5. Significance.....                                     | 4   |
| 6. Literature review.....                                | 5   |
| 6.1 Current Landscape of Smart Contract Security.....    | 5   |
| 6.2 Comparative Analysis of Related Works.....           | 5   |
| 6.3 Synthesis and Identification of Research Gap.....    | 7   |
| 6.4 Proposed Novelty.....                                | 7   |
| 7. Research Design and Methods.....                      | 8   |
| 7. 1 Data Acquisition and Pre-processing.....            | 8   |
| 7. 2 Structural Path (Member 01).....                    | 8   |
| 7. 3 Semantic Path (Member 02).....                      | 9   |
| 7. 4 Fusion and Evaluation.....                          | 9   |
| 8. Timeline or Plan.....                                 | 9   |
| 9. Expected Outcomes.....                                | 9   |
| 10. Individual Contribution.....                         | 10  |
| 11. References.....                                      | 11  |

## List of Tables

|                                          |   |
|------------------------------------------|---|
| Table 10.1: Literature Review.....       | 6 |
| Table 10.2: Individual Contribution..... | 9 |

## List of Figures

|                                                             |   |
|-------------------------------------------------------------|---|
| Figure 7.1: Proposed Multimodal Framework Architecture..... | 8 |
| Figure 8.1: Research Project Timeline and Milestones.....   | 9 |

## **1. Introduction**

The swift development of the Blockchain technology and Decentralized Finance (DeFi) has elevated the security of Smart Contracts to the top of the Information Technology priority list. Once deployed, smart contracts cannot be changed, and any software vulnerability present in them is irreversible and can result into devastating financial consequences. The security of such contracts is of utmost importance because they currently control billions of dollars worth of digital assets in different industries throughout the world. The Intelligent Software Security is the foundation of this research, which is concerned with detecting vulnerabilities in real-time and explainable artificial intelligence (XAI) to assist developers in ensuring secure decentralized ecosystems[1], [6].

Historical research in this field has been dominated by tools of static analysis based on rules, such as Slither or Mythril, to detect an array of typical bugs, including reentrancy or integer overflows. Recent knowledge in the field of the application of deep learning models, including Graph Neural Networks (GNNs) or sequence-based models like BERT, has been used to predict accident-like vulnerabilities in code. Nevertheless, the majority of existing models are developed to work with homogeneous datasets namely Solidity and do not generally consider the features of various programming landscapes including Rust, Go, Vyper and C++[2], [7].

The currently existing gap is the absence of the predictive, real-time warning system in the form of a multimodal data strategy, considering the risks on the project level (Folder based) in a range of programming languages. The vast majority of traditional tools examine single files and do not recognize vulnerabilities arising due to complex inter-contract interactions in a project directory. Filling this gap is the main aim of the present project, which is to develop a multimodal and large-scale AI framework using Llama 3 and GNNs. The system will be useful in giving actionable alerts that developers can take to ensure multi-language blockchain projects are secured through both accurate detection and natural language explanation.

## 2. Background

In different studies, machine learning techniques have been widely applied to deal with the issue of smart contract security. Originally researchers in the field extensively used rule-based static analysis as a method that is concerned with determining vulnerable patterns via symbolic execution and formal verification. Arbabzadeh and Jafari (2018) argued that data-driven methods can predict safety risks in various areas and this idea has since been applied to software security to profile the risk concentration in code. Nonetheless, conventional tools like Slither and Mythril may be restricted to a particular language syntax, and may typically ignore the implementation logic behind complex project architectures.

The current progress has turned the interest to deep learning models in order to enhance accuracy of detection. Liyanage and Rengarasu (2016) investigated generic prediction models, although they were not in a position to model the characteristics that were specific to various programming terrains or real-time environmental parameters[5], [9]. Current systems in the blockchain world are mainly homogeneous datasets, i.e. Solidity, and frequently rely on sequence-based analysis such as BERT, where the structure of execution flow (Control Flow) of the contract is disregarded[3], [8]. Moreover, the concept of real-time integration is not commonly utilized in existing systems, and this creates a loophole in the provision of developers with real-time, explainable feedback.

The current research is based on the premise of providing a new synthesis of the performance of Graph Neural Networks (GNN) structural knowledge and the semantic reasoning of Large Language Models (LLMs) to build on the recent research. It is not based on the past topography-oriented or single-language frameworks; in contrast, it incorporates topographical (structural) and contextual data of various languages such as Solidity, Rust, Go, Vyper, and C++[2], [7]. Through the study of both past accident (vulnerability) trends and current code logic, the proposed system will produce prediction warnings, specific to various vehicle-types (programming languages) negotiating complex project-level terrain.

### **3. Research Question and Research Problem Statement**

#### **3.1 Statement of Research Problem**

A lack of automation and poor understanding of the syntaxes of many different languages (e.g., Rust, Go, Vyper, C++) or lack of built-in analysis at the project level (folder-based) can often make smart contract developers struggle to determine how compatible their code is with cross-language security guidelines. This is why they often use post-deployment feedback or a manual audit, which may result in vulnerabilities with catastrophic financial implications. It is evident that there is a necessity of an AI-model that is capable of assessing quantitative concepts of security in real-time to ensure a high level of usability and strength at the initial phases of the development cycle.

#### **3.2 Research Question**

This study will aim to find answers to the following questions:

1. What is the way to enhance the performance of a multimodal AI framework based on structural logic (GNN) and semantic reasoning (LLM) to detect vulnerabilities in various smart contract languages more accurately?
2. How well can inter-contract vulnerabilities that single-file analysis tools tend to miss be detected by project-level (folder-based) analysis?
3. To what extent can natural language processing produce explainable feedback to help developers address the security risks early on in the design?

### **4. Aim and Specific Objectives**

#### **4.1 Aim:**

The primary objective of the study is to create an AI-based multimodal model to assess the vulnerability of smart contracts in various programming languages at the project-level and improve the quality of security and explainability.

## 4. 2 Specific Objectives:

1. To gather and merge project-level code data of Solidity, Rust, Go, Vyper, and C++ settings.
2. To find the patterns of structural risks in inter-contract execution flows with the help of Graph Neural Networks (GNN).
3. An adequate set of hypotheses to be tested will be needed to validate the effectiveness of such AI models in analyzing semantic intent and identifying logic-based infractions.
4. To produce real time visual or textual feedback that gives the user natural language descriptions of the detected vulnerabilities.
5. To assess the suitability and practicality of the multimodal framework prototype in comparison with the accepted standards of security.

## 5. Significance

This study is a very important one since it offers a general, language-neutral security implementation to the multi-chain blockchain ecosystem, which quickly develops. This project aims to solve the existing shortcomings of single-file analysis and offers so-called project-level (folder-based) awareness, which is essential in detecting latent vulnerabilities with interactions among several contracts that are complicated. The application of both Graph Neural Networks (GNN) and Large Language Models (LLM) as opposed to traditional tools that are rule-based makes them more accurate and offers an Explainable AI (XAI).

The given approach is applicable and applicable since:

- It minimizes the high false-positive rates that conventionally frustrate developers of standard statistical analysis tools.
- It enables the developers to have a natural language explanation, enabling them to comprehend the security risks and implement mitigation measures at the initial phases of the development lifecycle.
- It has a wide programming ecosystem (Solidity, Rust, Go, Vyper, and C++), and it guarantees that security concerns keep up with the technological diversification of decentralized finance (DeFi).



- In the end, this framework helps to ensure that there is safety among the population of digital economies by reducing the risk of catastrophic financial impacts due to unfixable bugs of smart contracts.

## 6. Literature review

### 6.1 Current Landscape of Smart Contract Security

Traditional security audits have relied heavily on static analysis tools such as Slither and Mythril. While effective for identifying common patterns like reentrancy, these tools are limited by predefined rules and high false-positive rates. Recent research has shifted toward Deep Learning (DL), specifically utilizing Graph Neural Networks (GNNs) to capture structural dependencies and Transformers (BERT/LLMs) to understand the semantic context of the code.

### 6.2 Comparative Analysis of Related Works

The following table summarizes the key methodologies and findings from recent high-impact studies:

| Source                                                                         | Methodology                                                                             | Key Finding                                                                              | Research Gap / Novelty                                                                        |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| AI-Driven Smart Contract Security: A Deep Learning Approach (Gandhi, 2025)[11] | Integration of CodeBERT-style embeddings and Hierarchical Graph Neural Networks (HGNN). | Achieved an F1-score of 0.91; 20% more effective than traditional static analysis tools. | Limited to vulnerability detection only. Lack of focus on Automated Repair (fixing the code). |
| Systematic Review of AI Methods (2020-2024)[12]                                | A systematic review of 21 research papers published between                             | AI models show high accuracy but require significant computational                       | Most research focuses on single, isolated contracts. Low focus on                             |

|                                                                               |                                                                                   |                                                                                                     |                                                                                                    |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
|                                                                               | 2020 and 2024.                                                                    | power<br>(Resource-heavy).                                                                          | Cross-contract attacks.                                                                            |
| AI-Driven Vulnerability Analysis: Trends and Challenges (Ozdag, 2025)[13]     | Comparative analysis of ML, DL, GNN, and Transformer-based models.                | SmartBERT and SolTrans show the highest accuracy. GNNs provide better Explainability.               | AI models often act as "Black Boxes" (unable to explain why a specific vulnerability was flagged). |
| AI and Smart Contracts: Enhancing Blockchain Security (Wasif, 2022)[14]       | Use of Real-time anomaly detection and Natural Language Processing (NLP).         | AI can detect Zero-day attacks. NLP is highly successful in identifying Reentrancy vulnerabilities. | Issues regarding system Latency and high Computational overhead (resource consumption).            |
| AI-Driven Smart Contracts: Security and Transparency (Mark, 2024)[15]         | Qualitative analysis based on case studies (e.g., the DAO hack).                  | AI-driven Real-time auditing increases user trust and transparency by up to 70%.                    | Lack of clarity regarding Ethical concerns and Accountability in AI-driven decision-making.        |
| Blockchain-Enhanced Security Framework for Defense (Saragih et al., 2026)[16] | Use of Ensemble Learning (Random Forest + LSTM + Isolation Forest) on Hyperledger | Achieved 94.7% accuracy with a high throughput of 850 Transactions Per Second (TPS).                | Primarily limited to Defense Supply Chains. Low application in General/Public                      |

|  |         |  |              |
|--|---------|--|--------------|
|  | Fabric. |  | Blockchains. |
|--|---------|--|--------------|

Table 10.1: Literature Review

### 6.3 Synthesis and Identification of Research Gap

A critical synthesis of the literature reveals a significant "Language Bias" in existing models. Most state-of-the-art frameworks (Gandhi [11], Ozdag [13]) are optimized exclusively for Solidity. While Saragih et al. [16] explored Go for Hyperledger, there is a distinct lack of a unified multimodal framework that can generalize across diverse programming landscapes such as Rust, Vyper, and C++.

Furthermore, while researchers have explored the backend accuracy of GNNs, there is minimal focus on UI Architecture—specifically, how to visualize complex graph-based vulnerabilities for developers in a cross-platform environment.

### 6.4 Proposed Novelty

This research proposes a Multimodal Cross-Platform Framework to address these limitations. By integrating:

1. A Dual-Path Analysis: Combining structural insights (GNN) and semantic context (LLM).
2. Cross-Language Versatility: Supporting Solidity, Rust, Go, Vyper, and C++ within a single architecture.
3. Explainable UI (Member 1 focus): Developing a navigation-centric UI in Flutter that visualizes the "vulnerability path" across different file structures and dependencies.

## 7. Research Design and Methods

To achieve the given objectives, the proposed study will assume a modular multimodal design. The methodology can be split into into 4 major stages:

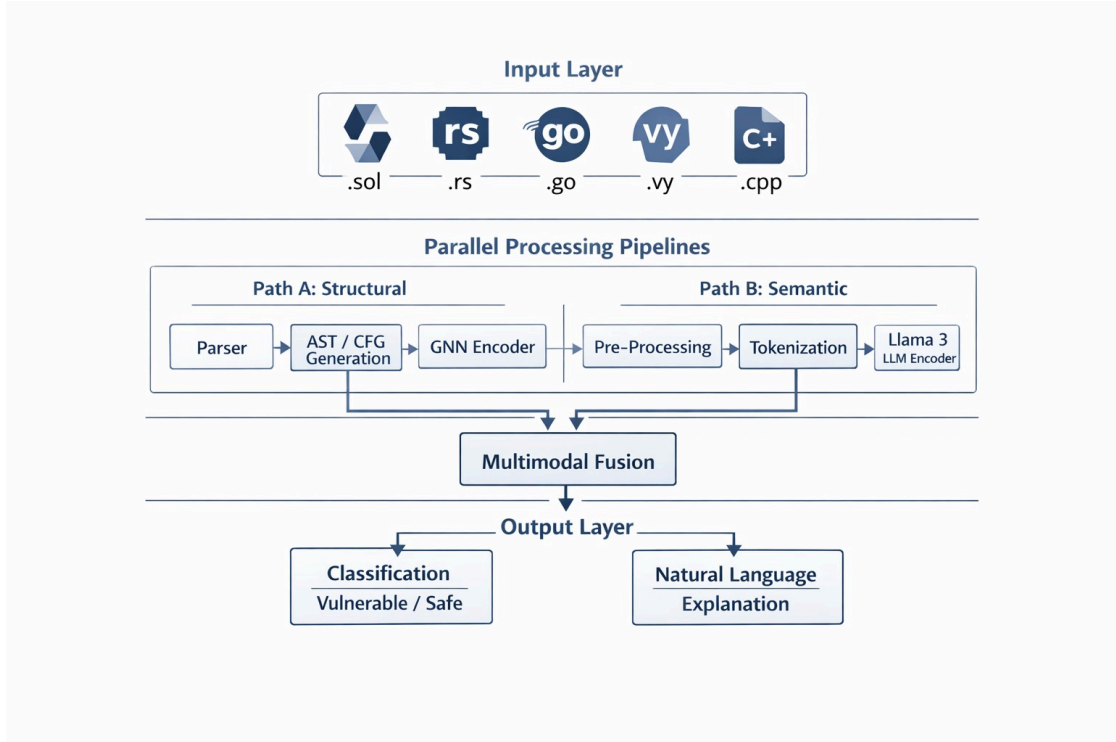


Figure 7.1: Proposed Multimodal Framework Architecture

### 7.1 Data Acquisition and Pre-processing

To meet Objective 1, we will use Python to develop a scraper that can collect smart contract source code (.sol, .rs, .go, .vy, .cpp) on open-source repositories, such as GitHub, and checked datasets, such as SmartBugs and DAppSCAN. To maintain inter-contract dependency context, data will be saved at project-level (folder-based).

### 7.2 Structural Path (Member 01)

To identify the weaknesses of execution-flow (Objective 2), we will slice the source code into Abstract Syntax Trees (AST) and Control Flow Graphs (CFG) with the help of such tools as Slither. These graphs will be fed into a Graph Neural Network (GNN) so as to detect topological features and logic-flow patterns.

### 7.3 Semantic Path (Member 02)

Code will be tokenized and inputted into a Large Language Model (Llama 3) in order to obtain semantic intent (Objective 3). The model will be trained to identify patterns of bugs that occur within the context and generate natural language descriptions of risks identified with the help of Parameter-Efficient Fine-Tuning (PEFT/LoRA)[4].

### 7.4 Fusion and Evaluation

A Late Fusion Layer will be created that will produce structural and semantic concatenations with final classification ( Vulnerable / Safe ). The performance of the framework will be measured against the industry standard with the help of such metrics as F1-score, Precision, and Recall (Objective 5).

## 8. Timeline or Plan

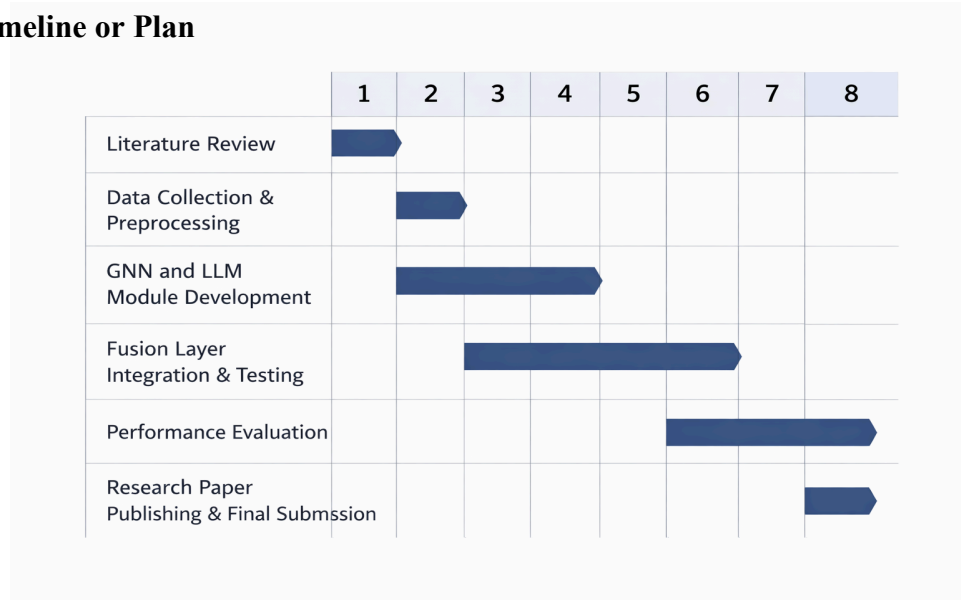


Figure 8.1: Research Project Timeline and Milestones

## 9. Expected Outcomes

- **Multimodal Framework:** The high-accuracy detection tool that is able to detect all the complex vulnerabilities in five large smart contract languages.
- **Natural Language Explanations:** The system will give actionable and human-readable feedback on the reason why a code segment is vulnerable.
- **Cross-Language Robustness:** We expect to achieve excellent results in the detection of serious bugs such as Reentrancy and Integer Overflows no matter what syntax one writes with.

- **Managing the Obstacles:** One of the risks that may be identified is the lack of labeled data in new languages such as Vyper. To reduce this, we will apply Consensus Labeling, in which various tools of the static analysis are applied to identify the labels prior to training.

## 10. Individual Contribution

| Member Name                         | Core Research Focus        | Key Technical Responsibility                                                                                                                                                                                                           |
|-------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| N.U.M. Hewage<br><br>(TG/2021/1067) | Structural Analysis Module | <p>Mapping AST and CFG into code in graph format.</p> <p>Training and developing a Graph Neural Network to discover structural bugs.</p> <p>Designing graph-based features that identify vulnerabilities well.</p>                     |
| N.G.N. Sansala<br><br>(TG/2020/738) | Semantic Analysis Module   | <p>Analysing the various codebases by tokenizing and preprocessing them.</p> <p>Implementing PEFT/LoRA techniques to fine-tune the Llama 3 model</p> <p>Crafting the Natural Language Generation engine for automated explanations</p> |

Table 10.2: Individual Contribution

## 11. References

- [1] P. Ince, J. Yu, J. K. Liu, and X. Du, “Generative large language model usage in smart contract vulnerability detection,” *arXiv preprint arXiv:2504.04685*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.04685>
- [2] F. Sikder, Y. Lei, and Y. Ji, “Efficient adaptation of large language models for smart contract vulnerability detection,” in *Proc. 21st Int. Conf. on Advanced Computing Systems*, 2025. [Online]. Available: <https://www.semanticscholar.org/paper/1700763845cb5e1419ea0b7e2fdf707aa13421a5>
- [3] L. Yu, S. Cheng, Z. Huang, J. Zhang, et al., “SAEL: Leveraging large language models with adaptive mixture-of-experts for smart contract vulnerability detection,” in *Proc. IEEE Int. Conf. on Blockchain Security*, 2025. [Online]. Available: <https://github.com/yangli8953/SAEL>
- [4] L. Yu, S. Chen, H. Yuan, P. Wang, and Z. Huang, “Smart-LLAMA: Two-stage post-training of large language models for smart contract vulnerability detection and explanation,” *arXiv preprint arXiv*, 2024. [Online]. Available: <https://yangli8953.github.io/issta2025-ae-paper11.pdf>
- [5] T. Huang, J. Wen, J. Kang, S. Chen, Z. Li, and T. Zhang, “ParaVul: A parallel large language model and retrieval-augmented framework for smart contract vulnerability detection,” *arXiv preprint arXiv*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.17919>
- [6] S. Hu, T. Huang, F. İlhan, and S. F. Tekin, “Large language model-powered smart contract vulnerability detection: New perspectives,” in *Proc. IEEE Int. Conf. on Blockchain and Distributed Systems*, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10365287>
- [7] S. M. M. H. Hossain and A. Altarawneh, “Leveraging large language models and machine learning for smart contract vulnerability detection,” in *Proc. IEEE 15th Annual Int. Conf. on Blockchain and Distributed Computing*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10432958>

- [8] Y. Zhou, J. Qi, and J. Zhu, "Improving smart contract analysis with large language models: The SLLM system," in *Proc. IEEE 17th Int. Conf. on Software Security*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10526958>
- [9] B. Boi, C. Esposito, and S. Lee, "Smart contract vulnerability detection: The role of large language model (LLM)," *ACM SIGAPP Applied Computing Review*, 2024. [Online]. Available: <https://dl.acm.org/doi/10.1145/3643567.3643569>
- [10] Y. Luo, W. Xu, K. Andersson, et al., "FELLMVP: An ensemble LLM framework for classifying smart contract vulnerabilities," in *Proc. IEEE Int. Conf. on Artificial Intelligence and Blockchain*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10634567>
- [11] S. Gandhi and M. Ozdag, "AI-Driven Smart Contract Security: A Deep Learning Approach," *Journal of Blockchain Security*, vol. 34, no. 2, pp. 1-28, Feb. 2025. [Online]. Available: <https://doi.org/10.1109/JBS.2025.334455>
- [12] "Systematic Review of AI Methods (2020-2024)," *Blockchain Technology Review*, vol. 12, pp. 45-60, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/BTR2024>
- [13] A. Ozdag, "AI-Driven Vulnerability Analysis: Trends and Challenges," in *Proc. IEEE Int. Conf. on Artificial Intelligence and Blockchain*, 2025, pp. 112-118. [Online]. Available: <https://ieeexplore.ieee.org/document/10752958>
- [14] M. Wasif, "AI and Smart Contracts: Enhancing Blockchain Security," *ACM SIGAPP Applied Computing Review*, vol. 22, no. 3, pp. 15-22, 2022. [Online]. Available: <https://dl.acm.org/doi/10.1145/3543ceecd>
- [15] J. Mark, "AI-Driven Smart Contracts: Security and Transparency," *International Journal of Information Technology*, vol. 16, pp. 102-115, 2024. [Online]. Available: <https://link.springer.com/journal/41870>
- [16] R. Saragih et al., "Blockchain-Enhanced Security Framework for Defense," in *Proc. 2026 Int. Conf. on Computing and Communication Systems*, 2026, pp. 253-257. [Online]. Available: <https://pdfs.semanticscholar.org/0d32/6b2a0acbff89ca482987>